Lab 02 — Images, layers and registries

Theory — how an image is named, what it is made of, where it comes from, and why Podman refuses to guess.

Objectives

- Break down a full image name and know what Docker fills in implicitly — and what Podman refuses to fill in.
- Understand why a **tag** is a moving label and a **digest** an identity.
- Explain the **layer** model and what it implies for disk and network.
- Know how to inspect an image without running it.
- Place Docker Hub, private registries and "official" images.

---

1. The full name of an image

You write `podman pull postgres:16-alpine`. The engine understands:

```
docker.io / library / postgres : 16-alpine
└registry┘  └namespace┘ └─repo──┘ └──tag──┘
```

Part	Role	Default value
registry	The server hosting the image	`docker.io` (Docker Hub)
namespace	The owning organisation or user	`library` (official images)
repository	The application name	*mandatory*
tag	The version	`latest`

Three immediate consequences:

- `postgres` alone means `docker.io/library/postgres:latest`.
- A company image carries a full name: `registry.mycompany.be/payments-team/billing-api:1.4.2`. A **dot** or a **port** in the first part signals a registry rather than a namespace.
- Images in the `library` namespace are the **official images**: maintained with Docker, audited, documented (`postgres`, `nginx`, `node`, `eclipse-temurin`). A `bobfrom59/postgres` image has none of those guarantees.

> **PODMAN**
> Docker completes `postgres` into `docker.io/library/postgres` without a word. Podman sees a risk there: a short name may point to a different image depending on the registry queried (*typosquatting*, a namesake on an internal registry). It applies `registries.conf`: known **aliases** (`alpine`, `nginx`, `postgres`…) resolved without a question, and for the rest the `unqualified-search-registries` list — if there are several, it asks you to choose. On Ubuntu that list only contains `docker.io`, so short names "work"; on Fedora or a `podman machine`, they trigger a question. A locally built image gets the **`localhost/`** prefix: `podman build -t api:1.0` produces `localhost/api:1.0`. `podman images` always shows the full name, no magic.

PITFALL
`latest` does not mean "the latest version". It is a **default** tag like any other, which the publisher chooses to move (or not); it may point at a two-year-old version. In production, `latest` is banned: non-reproducible deployment, no rollback.

2. Moving tag, immutable digest

A **tag** is a pointer: `postgres:16` means `16.10` today, `16.11` tomorrow. Nothing moves on your disk, but a new `pull` will bring something else. A **digest** is the SHA-256 fingerprint of the manifest: `postgres@sha256:9d0d1f1e…`. It is **computed from the content**, therefore:

- two images with the same digest are bit-for-bit identical, wherever they are;
- an image cannot change without its digest changing;
- a deployment can therefore be pinned absolutely.

→ SECURITY
SHA-256 is a **hash function**: it turns any data into 64 hexadecimal characters, deterministically (same input, same output) and one-way (impossible to craft an input that yields a chosen output). Changing a single bit of the image changes the whole fingerprint. It is the same principle that identifies a Git *commit*: an identifier that *is* a proof of integrity.

```
podman image inspect --format '{{.Digest}}' postgres:16-alpine
podman pull docker.io/library/postgres@sha256:9d0d1f1e...   # perfectly reproducible
```

The usual company compromise: one unique, immutable tag per build (`api:1.4.2` or `api:2026.03.17-b318`), never reused, and deployment tools that pin the digest.

3. An image is a stack of layers

Every build instruction that modifies the file system produces a **layer**: a set of files added, modified or removed compared with the previous state. The final image is the superposition of those layers, plus a manifest listing them and a configuration (default command, variables, user…).

```
┌──────────────────────────────┐  layer 4: COPY app.jar           (60 MB)
├──────────────────────────────┤  layer 3: the JRE                (180 MB)
├──────────────────────────────┤  layer 2: system packages        (30 MB)
├──────────────────────────────┤  layer 1: Debian slim base       (75 MB)
└──────────────────────────────┘

       ↑ read-only, shared between all the images that contain them
```

→ LINUX
The `overlay` storage driver is a kernel file system that **stacks** directories: "lower" read-only layers and one "upper" writable layer. Reading looks from top to bottom; writing first copies the file into the upper layer (*copy-on-write*); deleting creates a ghost file (*whiteout*) that hides without removing. The whole behaviour of images follows from those three rules.

This structure explains four behaviours you will see constantly:

1. Sharing on disk. If your twelve microservices start from the same JRE, those 180 MB are stored **once**. The sum of the `SIZE` column of `podman images` therefore far exceeds the space used

— `podman system df` gives the real figure.

2. Differential transfer. A `pull` or a `push` only transfers the missing layers: redeploying your API often transfers only the JAR layer.

3. Immutability, mistakes included. If a layer adds a password and a later layer deletes it, **the file is still in the image**: the later layer only hides it, and anyone with the image can recover it. A secret must never enter a build (lab 08).

4. The build cache. Since layers are identified by content, the engine reuses the ones it already has (lab 04).

> **REMEMBER**
> Layer sharing happens at the scale of the host or the registry, not of the image: an identical layer in two images is stored once. Rootless, that storage is in **your** `home` (`~/.local/share/containers/storage`): two users of the same machine share nothing.

> **PITFALL**
> A tag actually designates a **manifest list**, one per architecture (`linux/amd64`, `linux/arm64`…); the `pull` picks your machine's. An image built on an Apple Silicon MacBook therefore refuses to start on an `amd64` server: `exec format error`. `--platform` forces the architecture.

4. Everyday commands

```
podman pull nginx:alpine                  # download without running
podman images                             # list local images
podman images --filter dangling=true      # untagged images (orphaned layers)
podman history nginx:alpine               # the layers, their size and origin
podman image tree nginx:alpine            # the layers… as a tree, with their source image
podman image inspect nginx:alpine         # full metadata as JSON
podman tag nginx:alpine my-nginx:v1       # add a name (becomes localhost/my-nginx:v1)
podman rmi my-nginx:v1                    # remove a name (and the image if it was the last)
podman system df                          # space actually used
```

Two poorly understood subtleties:

`podman tag` copies nothing. It adds a label to the same image; both names point at the same `IMAGE ID`. Symmetrically, `rmi` on an image with two tags only removes the tag: the data only goes when the last name disappears.

A "dangling" image (`<none>:<none>`) is not a mysterious leftover. It is an image whose tag was moved to a newer version: it lost its name but still takes disk space — the normal residue of successive rebuilds.

Getting an image out of the engine

```
podman save -o api.tar my-api:1.0                        # archive in docker-archive format
podman save --format oci-archive -o api.tar my-api:1.0   # same thing, in the standard OCI format
podman load -i api.tar                                   # re-imports the image, tags included
```

Useful when the target has no access to the registry (isolated site). Not to be confused with `export` / `import`, which flatten the file system **of a container** and lose layers and configuration (`CMD`, `ENV`, `EXPOSE`...).

5. Registries

A registry is an HTTP service that stores layers and manifests behind a standardised API (`/v2/…`) that every tool speaks.

> **→ HTTP**
> A REST API exposes *resources* at URLs and manipulates them with HTTP verbs: `GET /v2/_catalog` lists the repositories, `HEAD /v2/api/manifests/1.0` returns the digest in a header, `PUT` pushes a layer. `curl` is therefore enough to query a registry — you will do it in the hands-on lab. It is the mechanics of a Spring Boot API.

Type	Examples	Use
Public	Docker Hub, `ghcr.io`, `quay.io`	Base images and off-the-shelf software
Private, managed	AWS ECR, Azure ACR, Google AR	In-house images, hosted by the cloud
Private, self-hosted	Harbor, Nexus, GitLab Registry	In-house images, full control, vulnerability scanning

The company cycle:

```
podman login registry.mycompany.be
podman tag api:1.4.2 registry.mycompany.be/payments/api:1.4.2
podman push registry.mycompany.be/payments/api:1.4.2
```

Three things to know:

- **`podman login` stores the token** in `${XDG_RUNTIME_DIR}/containers/auth.json`, a temporary file wiped at logout — where Docker writes to `~/.docker/config.json`, in the clear (base64) and for ever. On a CI agent, ephemeral credentials are preferred.
- **Podman requires TLS.** A plain-HTTP registry — like the one you will start on `localhost:5000` — is refused (`http: server gave HTTP response to HTTPS client`) until you say `--tls-verify=false` or declare the registry `insecure = true` in `registries.conf`. Docker makes a silent exception for `localhost`; Podman does not.
- **Docker Hub rate-limits anonymous downloads** (quota per IP): on a CI this yields `toomanyrequests`, hence the use of a *pull-through cache* or an internal copy of the base images.

6. In the workplace

On a Spring Boot + Angular stack:

- The **base images** (`eclipse-temurin`, `node`, `nginx`, `postgres`) are copied into the internal registry, often with `skopeo copy` — Podman's sister tool that copies from one registry to another without downloading anything. Nobody pulls from the Internet in production: quota, availability, control of what comes in.

- CI builds `registry.internal/myapp/api:<version>` and `…/web:<version>`, then pushes; the version comes from the Git tag or the build number. A **scanner** (Trivy, Harbor, Grype) blocks images carrying critical vulnerabilities. Deployment references a precise version, never `latest`.

Remember

- A full name is `registry/namespace/repository:tag`; by default `docker.io`, `library` and `latest`. Podman always displays that full name and prefixes your builds with `localhost/`.
- `latest` is not "the most recent": it is a default tag, to be banned in production.
- The **tag** can move, the **digest** `sha256:…` identifies exact content.
- An image is a stack of **read-only layers**, shared between images, transferred differentially — and a file deleted in a later layer stays there: never a secret in a build.
- `tag` duplicates nothing; `rmi` first removes a name, not data. Podman requires TLS: `--tls-verify=false` only for a local test registry.
- `save` / `load` carry a complete image; `export` / `import` flatten a container and lose its configuration.

Vocabulary

repository: the versions of an image. — **tag**: moving label. — **digest**: immutable SHA-256 fingerprint. — **manifest**: description of the layers and configuration; the **manifest list** indexes several architectures. — **dangling image**: image that lost its tag. — **layer**: file layer. — **overlay**: driver that stacks the layers. — **pull-through cache**: local mirror of a public registry. — **official image**: `library` namespace on Docker Hub. — **short name**: name without registry, resolved by `registries.conf`. — **skopeo**: copying and inspecting images between registries.